PROJECT 11 — B.Tech Second Semester

# Global Energy Risk Monitor System

## Complete Team Implementation Plan

Written for students with basic C++ knowledge only

| 6 | 27 | 5 |
| --- | --- | --- |
| Team Members | Clear Tasks | Weeks to Build |

# SECTION 1

How To Read This Plan

## What is this document?

This is your complete step-by-step guide to build the Global Energy Risk Monitor System in C++. Every task is written in plain language so you can follow it without needing to know anything beyond basic C++. Each task description is also designed so you can paste it directly into any AI coding tool to get working code.

**How to use this plan with an AI tool (like Google AI, ChatGPT, etc.)**

Find your task. Read the task description box. Copy the description. Paste it into the AI tool. The AI writes the code. You read it, understand it, and save it into your file.

## What you will build

A console-based C++ program (runs in the terminal, no graphics needed) that does all of this:

- Reads energy resource data from a CSV file saved on your computer
- Reads geopolitical event data (wars, sanctions) from another CSV file
- Calculates a risk score from 0 to 100 for each energy resource
- Labels each resource as LOW, MEDIUM, or HIGH risk
- Shows a menu so users can search, compare, and view summaries
- Saves results to an output file when the program exits

**You do NOT need to build:**

No website. No app. No graphics. Just a clean, working C++ console program — exactly what a second-semester student should build.

## Tools you need — and nothing else

| Tool | What it does | Easy to set up? |
|---|---|---|
| C++ (you already know this) | The programming language for everything | Yes — you already have it |
| VS Code or Code::Blocks | To write and run your code | Yes — free download |
| g++ compiler | Turns your code into a running program | Yes — one install command |
| Notepad or any text editor | To create your .csv data files | Yes — already on your computer |

# SECTION 2

Team Roles — Who Does What

## How the 6 members divide the work

Each member owns exactly one part of the project. This means no one steps on each other's work. When you put everything together in Week 5, all the pieces will connect.

### Member 1 — Team Leader + Main Menu + Data Files

- Creates the project folder structure everyone uses
- Creates energy_data.csv and events.csv with sample data
- Writes main.cpp with the menu (options 1 to 6)
- Connects all modules together in Week 5
- Tests the full program end to end
- Prepares the final report and demo for the professor

### Member 2 — Data Manager — File Reading and Writing

- Writes data_loader.h with EnergyResource and GeopoliticalEvent structs
- Writes data_loader.cpp that reads both CSV files
- Handles errors when a file cannot be opened
- Writes the function that saves risk scores to an output file
- Tests file reading by printing loaded data to the terminal

### Member 3 — Risk Engineer — Score Calculator

- Writes risk_engine.h with the RiskScore struct
- Writes 5 sub-score functions (supply, conflict, trade, demand, route)
- Writes calculateRisk() that combines all sub-scores into one final score
- Classifies each score as LOW, MEDIUM, or HIGH
- Tests scoring with known values and verifies the output is correct

### Member 4 — Feature Builder A — Resource Lookup and Events

- Writes features.h with function declarations
- Implements searchResource() — find a resource by name and show its risk
- Implements showActiveEvents() — list all current geopolitical events
- Implements showEventsForRegion() — filter events by country/region

### Member 5 — Feature Builder B — Supply Analysis and Region Comparison

- Writes analysis.h with function declarations
- Implements analyzeSupplyDisruption() — shows which resources are most at risk

- Implements compareRegionRisk() — ranks regions from highest to lowest risk
- Formats output as a readable table in the terminal

### Member 6 — Display Engineer — Output Formatting and Summary

- Writes display.h with function declarations
- Implements printRiskTable() — sorted table of all resources and scores
- Implements displayGlobalSummary() — overall risk index and top risks
- Implements printRiskLabel() — makes HIGH/MEDIUM/LOW look distinct on screen

# SECTION 3

Folder Structure — How to Organise Your Files

## Create this exact folder structure — all 6 members must use the same layout

```
EnergyRiskMonitor/                  <- Your main project folder
  |
  |-- main.cpp                      <- Main menu (Member 1)
  |
  |-- data_loader.h                 <- Struct definitions + function declarations
(Member 2)
  |-- data_loader.cpp               <- CSV reading and file writing (Member 2)
  |
  |-- risk_engine.h                 <- RiskScore struct + function declaration (Member
3)
  |-- risk_engine.cpp               <- All score calculation code (Member 3)
  |
  |-- features.h                    <- Feature function declarations (Member 4)
  |-- features.cpp                  <- Search + event viewing (Member 4)
  |
  |-- analysis.h                    <- Analysis function declarations (Member 5)
  |-- analysis.cpp                  <- Supply + region analysis (Member 5)
  |
  |-- display.h                     <- Display function declarations (Member 6)
  |-- display.cpp                   <- All terminal output formatting (Member 6)
  |
  |-- data/                         <- Folder for all data files
       |-- energy_data.csv          <- Energy resources (Member 1 creates this)
       |-- events.csv               <- Geopolitical events (Member 1 creates this)
       |-- risk_scores_output.csv   <- Generated automatically when program exits
```

## How to set up — step by step

1. Create a folder on your desktop called EnergyRiskMonitor
2. Inside that folder, create another folder called data
3. Open VS Code and open the EnergyRiskMonitor folder as your workspace
4. Create all .cpp and .h files shown above — leave them empty for now
5. Member 1 shares the folder structure with the team via zip or Google Drive

## How to compile the whole program — one command

```
g++ main.cpp data_loader.cpp risk_engine.cpp features.cpp analysis.cpp display.cpp -o
EnergyRisk -std=c++11
```

Run this command inside your EnergyRiskMonitor folder. It compiles all files together into one program called EnergyRisk.

# SECTION 4

Data Files — What to Store and How

## File 1 — energy_data.csv

This is a plain text file. Each line is one energy resource. Values are separated by commas. The first line is the header (column names). Member 1 creates this file.

```
id,name,type,region,production,consumption,reserve_years,export_dependency,price
OIL_SAU,Saudi Crude Oil,Oil,Saudi Arabia,12.0,3.5,65,0.85,82.40
OIL_RUS,Russian Crude Oil,Oil,Russia,11.0,3.8,28,0.72,78.10
GAS_RUS,Russian Natural Gas,Gas,Russia,638.0,420.0,55,0.68,9.50
GAS_QAT,Qatari LNG,Gas,Qatar,180.0,32.0,80,0.90,11.20
OIL_IRN,Iranian Crude Oil,Oil,Iran,3.8,1.9,90,0.60,74.00
OIL_IRQ,Iraqi Crude Oil,Oil,Iraq,4.5,0.9,50,0.78,80.50
OIL_USA,US Crude Oil,Oil,USA,13.2,19.8,12,0.20,81.00
GAS_NOR,Norwegian Natural Gas,Gas,Norway,117.0,5.0,35,0.92,10.80
```

## File 2 — events.csv

This file stores geopolitical events. Member 1 creates this file. is_active = 1 means the event is currently happening.

```
id,title,type,region,intensity,supply_impact,is_active
E001,Russia-Ukraine War,War,Russia,0.85,0.30,1
E002,US-Iran Sanctions,Sanctions,Iran,0.70,0.45,1
E003,OPEC Production Cut,ProductionCut,Saudi Arabia,0.55,0.15,1
E004,Libyan Political Instability,Instability,Libya,0.60,0.35,1
E005,Red Sea Trade Disruption,TradeRestriction,Yemen,0.65,0.25,1
```

## What each column means

| Column | What it means | Example |
|---|---|---|
| id | Unique short code for the resource or event | OIL_SAU |
| name / title | Full readable name | Saudi Crude Oil |
| type | Category: Oil, Gas, Electricity / War, Sanctions, etc. | Oil |
| region | Country or region name | Saudi Arabia |
| production | How much is produced per day | 12.0 |
| consumption | How much is used per day | 3.5 |
| reserve_years | Years of reserves remaining | 65 |
| export_dependency | 0.0 to 1.0 — how much others depend on this source | 0.85 |
| price | Current price in USD | 82.40 |
| intensity | Event severity: 0.0 = weak, 1.0 = very severe | 0.85 |

| | | |
|---|---|---|
| supply_impact | How much supply is reduced: 0.0 to 1.0 | 0.30 |
| is_active | 1 = happening now, 0 = ended | 1 |

# SECTION 5

Risk Score Formula — How to Calculate It

## The 5 factors and their weights

The risk score is a number from 0 to 100. It is made up of 5 sub-scores, each multiplied by a weight. Higher score = more dangerous.

| Factor | Weight | Simple formula to calculate it (result: 0 to 100) |
|---|---|---|
| Supply Disruption | 30% | If consumption > production: ((consumption / production) - 0.8) / 0.7 x 100. If not, score = 5. Cap at 100. |
| Conflict Intensity | 25% | Add up intensity x 100 for every active War or Instability event in the same region as the resource. |
| Trade Restrictions | 20% | Add up intensity x 100 for every active Sanctions or TradeRestriction event that affects this region. |
| Demand Pressure | 15% | export_dependency x 50. Then add: if reserve_years < 30, also add (30 - reserve_years) / 30 x 50. |
| Route Vulnerability | 10% | Simple lookup by region: Russia = 60, Iran = 55, Libya = 50, Saudi Arabia = 15, all others = 10. |

**Final formula**

Risk Score = (Supply x 0.30) + (Conflict x 0.25) + (Trade x 0.20) + (Demand x 0.15) + (Route x 0.10)
Score below 33 -> LOW risk Score 33 to 66 -> MEDIUM risk Score above 66 -> HIGH risk

## Worked example — Russian Crude Oil

| Factor | Calculation | Score |
|---|---|---|
| Supply | consumption(3.8) / production(11.0) = 0.35 — below 0.8 threshold, so score = 5 | 5 |
| Conflict | Russia-Ukraine War: intensity 0.85 x 100 = 85 | 85 |
| Trade | EU Sanctions: intensity 0.70 x 100 = 70 | 70 |
| Demand | export_dependency 0.72 x 50 = 36 | 36 |
| Route | Russia region lookup = 60 | 60 |
| FINAL | (5 x 0.30) + (85 x 0.25) + (70 x 0.20) + (36 x 0.15) + (60 x 0.10) | = 48.15 -> MEDIUM |

# SECTION 6

Phase-by-Phase Tasks — What to Build and When

PHASE 1

## Setup, Data Files, and File Reading

Week 1 — Members 1 and 2

**Member 1 tasks:**

TASK 1.1

### Create the project folder structure

Create the EnergyRiskMonitor folder and the data/ subfolder on your computer. Create all the empty .cpp and .h files listed in Section 3 (leave them blank). Share the structure with your team via a zip file or Google Drive so everyone starts with the same layout.

TASK 1.2

### Create energy_data.csv

Using VS Code or Notepad, create the file data/energy_data.csv. Type the header row first, then add at least 8 energy resources exactly as shown in Section 4. Save it as plain text. Double-check that you have no extra spaces around the commas.

TASK 1.3

### Create events.csv

Create data/events.csv. Add at least 5 active geopolitical events as shown in Section 4. Make sure is_active is set to 1 for current events.

TASK 1.4

### Write main.cpp with a menu

Write main.cpp. It should display a menu when the program starts: 1) View all resources and risk scores 2) Search for a resource by name 3) View active geopolitical events 4) Compare regions by risk 5) View global risk summary 6) Save results and exit Do not implement the features yet. Just show the menu, read the user's choice, and print 'Feature coming soon.' for each option. Use a while loop so the menu keeps appearing until the user picks 6.

**Member 2 tasks:**

TASK 1.5

### Write data_loader.h — struct definitions

Write data_loader.h. Define two structs: EnergyResource with these fields: string id, name, type, region; double production, consumption, reserve_years, export_dependency, price. GeopoliticalEvent with these fields: string id, title, type, region; double intensity, supply_impact; int is_active. Also declare (do not write yet) two functions: vector<EnergyResource> loadEnergyResources(string filename); vector<GeopoliticalEvent> loadEvents(string filename);

TASK 1.6

### Write data_loader.cpp — CSV file reading

Write data_loader.cpp. Implement loadEnergyResources(): open the file using ifstream, skip the first line (header) using getline(), then for each remaining line use getline() with comma delimiter to read each field into an EnergyResource struct. Push each struct into a

vector<EnergyResource> and return it. Do the same for loadEvents() reading events.csv. If a file cannot be opened, print an error message and return an empty vector.

TASK 1.7

### Test that file reading works

In main.cpp, temporarily call loadEnergyResources() and use a for loop to print the name and region of every loaded resource. Run the program and confirm you can see all 8 resources printed. Fix any bugs before moving to Phase 2. This test can be removed after Phase 2 begins.

PHASE 2

## Risk Score Engine

Week 2 — Member 3

**Member 3 tasks:**

TASK 2.1

### Write risk_engine.h — RiskScore struct and function declaration

Write risk_engine.h. Define a struct called RiskScore with these fields: string resource_id, resource_name, region, level; double raw_score, supply_score, conflict_score, trade_score, demand_score, route_score; Also declare this function: RiskScore calculateRisk(EnergyResource r, vector<GeopoliticalEvent> events);

TASK 2.2

### Write the 5 sub-score functions in risk_engine.cpp

Write 5 small functions — one for each factor. Each takes the resource and/or events as input and returns a double between 0 and 100. Supply score: if production > 0, calculate ((consumption/production) - 0.8) / 0.7 x 100. Use max(0, ...) and min(100, ...) to keep it in range. If production is 0 return 100. Conflict score: loop through events. For each event where is_active==1 and region matches and type is War or Instability, add intensity x 100 to a total. Return the total (capped at 100). Trade score: same as conflict but for type Sanctions or TradeRestriction. Demand score: export_dependency x 50. If reserve_years < 30, also add (30 - reserve_years) / 30 x 50. Route score: use if-else to check region name and return hardcoded value: Russia=60, Iran=55, Libya=50, Saudi Arabia=15, default=10.

TASK 2.3

### Write calculateRisk() — combine all sub-scores

Write the main calculateRisk() function. Call each of the 5 sub-score functions. Combine them: raw = supply x 0.30 + conflict x 0.25 + trade x 0.20 + demand x 0.15 + route x 0.10 Fill in a RiskScore struct with all values. Set the level field: if raw > 66 set level = HIGH, else if raw > 33 set level = MEDIUM, else set level = LOW. Return the filled RiskScore.

TASK 2.4

### Test with the Russian Crude Oil example

In main.cpp temporarily create one EnergyResource (Russian Crude Oil with the exact values from Section 4) and one GeopoliticalEvent (Russia-Ukraine War, intensity 0.85). Call calculateRisk() and print raw_score and level to the terminal. The answer should be close to 48.15 and level should be MEDIUM. Fix any errors before Phase 3.

PHASE 3

## Feature Modules

Week 3 — Members 4 and 5

**Member 4 tasks:**

TASK 3.1

### Write features.h

Write features.h. Include data_loader.h and risk_engine.h at the top. Declare these three functions: void searchResource(vector<EnergyResource> resources, vector<GeopoliticalEvent> events); void showActiveEvents(vector<GeopoliticalEvent> events); void showEventsForRegion(vector<GeopoliticalEvent> events, string region);

TASK 3.2

### Implement searchResource() in features.cpp

Ask the user to type a resource name or ID (use cin). Loop through the resources vector and find a match. If found: call calculateRisk() and print all details: name, type, region, production, consumption, price, each sub-score, final score, and risk level. If not found, print: Resource not found. Please check the name and try again.

TASK 3.3

### Implement showActiveEvents()

Loop through all events. If is_active == 1, print the event details on screen: title, type, region, intensity as a percentage (intensity x 100), and supply impact as a percentage (supply_impact x 100). Print a line of dashes between events to separate them. At the end print the total count of active events.

TASK 3.4

### Implement showEventsForRegion()

The region name is passed as a parameter. Loop through events and print only those where region matches and is_active == 1. If no matches are found, print: No active events found for this region.

**Member 5 tasks:**

TASK 3.5

### Write analysis.h

Write analysis.h. Include data_loader.h and risk_engine.h. Declare these two functions: void analyzeSupplyDisruption(vector<EnergyResource> resources, vector<GeopoliticalEvent> events); void compareRegionRisk(vector<EnergyResource> resources, vector<GeopoliticalEvent> events);

TASK 3.6

### Implement analyzeSupplyDisruption()

For each resource, calculate its risk score. Sort the results from highest supply_score to lowest. Print a list showing: resource name, region, supply score, and the name of the event causing the disruption (if any active event affects that region). At the end print: X out of Y resources are currently at HIGH supply disruption risk.

TASK 3.7

### Implement compareRegionRisk()

Group resources by region name. For each region, calculate the average raw_score across all its resources. Sort regions from highest to lowest average score. Print a formatted table with columns: Region Name, Number of Resources, Average Risk Score, Risk Level. Use setw() from <iomanip> to align the columns neatly.

PHASE

## Display, Summary, and File Output

## 4 Week 4 — Member 6 and Member 2

**Member 6 tasks:**

TASK 4.1

### Write display.h

Write display.h. Include data_loader.h and risk_engine.h. Declare these functions: void printRiskTable(vector<EnergyResource> resources, vector<GeopoliticalEvent> events); void displayGlobalSummary(vector<EnergyResource> resources, vector<GeopoliticalEvent> events); void printRiskLabel(string level);

TASK 4.2

### Implement printRiskTable()

Calculate risk scores for all resources. Sort them from highest to lowest raw_score. Print a formatted table with these columns using setw() from <iomanip>: Resource Name (20 wide) | Region (15 wide) | Type (12 wide) | Score (8 wide) | Risk Level Print a header row and a separator line of dashes before the data rows.

TASK 4.3

### Implement displayGlobalSummary()

Calculate and display all of the following: 1) Total number of resources in the system 2) Global risk index = average of all raw_score values 3) Total number of active events 4) Count of HIGH, MEDIUM, and LOW risk resources 5) A simple bar made of = signs for each category: HIGH [========] 4 resources MEDIUM [=====] 3 resources LOW [==] 2 resources 6) The top 3 highest-risk resource names with their scores

TASK 4.4

### Implement printRiskLabel()

This function takes a string (HIGH, MEDIUM, or LOW) and prints it with special formatting: HIGH -> print: [!! HIGH !!] MEDIUM -> print: [~ MEDIUM ~] LOW -> print: [ LOW ] Call this function whenever you display a risk level throughout the program.

**Member 2 additional task:**

TASK 4.5

### Implement saveRiskScores() in data_loader.cpp

Add a new function to data_loader.cpp: void saveRiskScores(vector<RiskScore> scores, string filename). It opens the file for writing using ofstream. Write a header row first: resource_id,resource_name,region,raw_score,level,supply_score,conflict_score,trade_score,demand_score,route_score. Then write one line per RiskScore struct. This function is called from main.cpp when the user picks option 6 (Exit).

## PHASE 5 Integration, Testing, and Final Polish

Week 5 — All Members

**Member 1 tasks:**

TASK 5.1

### Connect all modules in main.cpp

Replace the 'Feature coming soon' placeholders with real function calls: Option 1 -> call printRiskTable(resources, events) Option 2 -> call searchResource(resources, events) Option 3 -> ask user for region name, call showEventsForRegion(events, region) Option 4 -> call compareRegionRisk(resources, events) Option 5 -> call displayGlobalSummary(resources, events) Option 6 -> call saveRiskScores(scores, filename) then exit the loop Make sure loadEnergyResources() and loadEvents() are called once at the start of main() before the menu loop begins.

TASK 5.2

### Full system test — test every single option

Run the full program. Test every menu option one by one: - Option 2: search for a resource that exists (e.g. OIL_RUS) - Option 2: search for a resource that does NOT exist - Option 3: search for events in Russia (should find results) - Option 3: search for events in Japan (should say no events found) - Option 6: check that risk_scores_output.csv is created in the data folder Fix every bug found before submitting.

**All member tasks:**

TASK 5.3

### Add more data to the CSV files

Each member adds at least one new energy resource and one new event to the data files. Suggestions: EU Electricity, Chinese Coal, Venezuelan Oil, US Natural Gas, North Sea Oil, Norwegian Wind Energy. The more data, the more impressive the demo will look.

TASK 5.4

### Write comments in all your code files

Go through your own .cpp and .h files. Above every function, add a comment in plain English explaining what the function does. Above every important variable, add a short comment. Example: // This function reads energy_data.csv and returns a list of all resources vector<EnergyResource> loadEnergyResources(string filename) This is required for your professor's evaluation.

# SECTION 7

AI Tool Prompts — Copy and Paste These

For each task below, copy the entire prompt and paste it into any AI coding tool. The AI will write the code. You then read it, check it makes sense, and save it into your file.

## Prompt for Member 2 — Tasks 1.5 and 1.6 (File Reading)

**Copy this entire block and paste into your AI tool:**

```
Write two C++ files: data_loader.h and data_loader.cpp.

In data_loader.h define two structs:
  struct EnergyResource {
    string id, name, type, region;
    double production, consumption, reserve_years, export_dependency, price;
  };
  struct GeopoliticalEvent {
    string id, title, type, region;
    double intensity, supply_impact;
    int is_active;
  };

In data_loader.cpp implement:
  vector<EnergyResource> loadEnergyResources(string filename)
  vector<GeopoliticalEvent> loadEvents(string filename)

Both functions open a CSV file with ifstream, skip the first line (header),
then use getline with comma as delimiter to read each field.
If the file cannot be opened, print an error and return an empty vector.
Use only standard C++ libraries. No external libraries.
```

## Prompt for Member 3 — Tasks 2.1 to 2.3 (Risk Engine)

**Copy this entire block and paste into your AI tool:**

```
Write two C++ files: risk_engine.h and risk_engine.cpp.
Assume EnergyResource and GeopoliticalEvent structs are already defined
in data_loader.h with the fields listed below.

EnergyResource fields: string id, name, type, region;
  double production, consumption, reserve_years, export_dependency, price.
GeopoliticalEvent fields: string id, title, type, region;
  double intensity, supply_impact; int is_active.

In risk_engine.h define:
  struct RiskScore {
    string resource_id, resource_name, region, level;
    double raw_score, supply_score, conflict_score,
           trade_score, demand_score, route_score;
  };
  RiskScore calculateRisk(EnergyResource r, vector<GeopoliticalEvent> events);

In risk_engine.cpp implement calculateRisk() using these sub-scores:
```

```
  supply_score:   if production>0: min(100, max(0, ((consumption/production)-
0.8)/0.7*100))
  conflict_score: sum of intensity*100 for active (is_active==1) War/Instability
                  events where region matches resource region. Cap at 100.
  trade_score:    sum of intensity*100 for active Sanctions/TradeRestriction
                  events where region matches. Cap at 100.
  demand_score:   export_dependency*50 + (if reserve_years<30: (30-
reserve_years)/30*50)
  route_score:    if region==Russia: 60, Iran: 55, Libya: 50, Saudi Arabia: 15, else:
10

  raw_score = supply*0.30 + conflict*0.25 + trade*0.20 + demand*0.15 + route*0.10
  level: HIGH if raw>66, MEDIUM if raw>33, else LOW

Use only standard C++. No external libraries.
```

## Prompt for Member 4 — Tasks 3.1 to 3.4 (Resource Lookup and Events)

**Copy this entire block and paste into your AI tool:**

```
Write two C++ files: features.h and features.cpp.
Assume EnergyResource, GeopoliticalEvent, and RiskScore structs are defined
in data_loader.h and risk_engine.h. Assume calculateRisk() is available.

Implement these three functions:

1) void searchResource(vector<EnergyResource> resources,
                       vector<GeopoliticalEvent> events)
   Ask user to type a resource name or id with cin.
   Search the vector for a match. If found: calculate risk with calculateRisk()
   and print all resource fields and all RiskScore fields in a readable format.
   If not found: print 'Resource not found.'

2) void showActiveEvents(vector<GeopoliticalEvent> events)
   Print all events where is_active==1.
   For each: print title, type, region, intensity as percentage, supply impact as
percentage.
   Print a separator line between each event.
   At the end print the total count of active events.

3) void showEventsForRegion(vector<GeopoliticalEvent> events, string region)
   Print events where region matches and is_active==1.
   If none found print: No active events found for this region.

Use only standard C++. No external libraries.
```

## Prompt for Member 5 — Tasks 3.5 to 3.7 (Analysis)

**Copy this entire block and paste into your AI tool:**

```
Write two C++ files: analysis.h and analysis.cpp.
Assume EnergyResource, GeopoliticalEvent, and RiskScore structs are defined
in data_loader.h and risk_engine.h. Assume calculateRisk() is available.

Implement these two functions:

1) void analyzeSupplyDisruption(vector<EnergyResource> resources,
                                vector<GeopoliticalEvent> events)
```

```
   For each resource calculate its RiskScore using calculateRisk().
   Sort results from highest to lowest supply_score.
   Print each: resource name, region, supply_score, and any active
   event title that affects that region.
   At the end print: 'X of Y resources at HIGH supply risk.'

2) void compareRegionRisk(vector<EnergyResource> resources,
                          vector<GeopoliticalEvent> events)
   Group resources by region. For each region calculate average raw_score.
   Sort regions from highest to lowest average score.
   Print a table with columns: Region (20 wide), Resources (12 wide),
   Avg Score (12 wide), Risk Level (12 wide).
   Use setw() from <iomanip> for column alignment.

Use only standard C++. No external libraries.
```

## Prompt for Member 6 — Tasks 4.1 to 4.4 (Display)

**Copy this entire block and paste into your AI tool:**

```
Write two C++ files: display.h and display.cpp.
Assume EnergyResource, GeopoliticalEvent, and RiskScore structs are defined.
Assume calculateRisk() is available from risk_engine.h.

Implement these functions:

1) void printRiskTable(vector<EnergyResource> resources,
                       vector<GeopoliticalEvent> events)
   Calculate risk for all resources. Sort by raw_score descending.
   Print a table with header: Name(20) | Region(15) | Type(12) | Score(8) | Level
   Use setw() from <iomanip>. Print a dashes separator after the header.

2) void displayGlobalSummary(vector<EnergyResource> resources,
                             vector<GeopoliticalEvent> events)
   Print: total resources, global average risk score, total active events,
   count of HIGH/MEDIUM/LOW resources, a bar chart using = signs for each level,
   and the top 3 resource names with highest raw_score.

3) void printRiskLabel(string level)
   If level==HIGH print:   [!! HIGH !!]
   If level==MEDIUM print: [~ MEDIUM ~]
   If level==LOW print:    [  LOW  ]

Use only standard C++. Include <iomanip> for setw(). No external libraries.
```

# SECTION 8

Master Task Tracker — All 27 Tasks

Print this table. Tick off each task as your team completes it.

| Task | Description | Member | Phase | Done? |
|---|---|---|---|---|
| 1.1 | Create project folder structure + all empty files | M1 | 1 | ☐ |
| 1.2 | Create energy_data.csv with 8+ energy resources | M1 | 1 | ☐ |
| 1.3 | Create events.csv with 5+ events | M1 | 1 | ☐ |
| 1.4 | Write main.cpp with basic menu (no feature code yet) | M1 | 1 | ☐ |
| 1.5 | Write data_loader.h — both struct definitions | M2 | 1 | ☐ |
| 1.6 | Write data_loader.cpp — loadEnergyResources() and loadEvents() | M2 | 1 | ☐ |
| 1.7 | Test file reading — print all loaded resources to terminal | M2 | 1 | ☐ |
| 2.1 | Write risk_engine.h — RiskScore struct + function declaration | M3 | 2 | ☐ |
| 2.2 | Write 5 sub-score functions in risk_engine.cpp | M3 | 2 | ☐ |
| 2.3 | Write calculateRisk() combining all sub-scores | M3 | 2 | ☐ |
| 2.4 | Test calculateRisk() with Russian Crude Oil — verify score ~48 | M3 | 2 | ☐ |
| 3.1 | Write features.h — declare 3 feature functions | M4 | 3 | ☐ |
| 3.2 | Implement searchResource() | M4 | 3 | ☐ |
| 3.3 | Implement showActiveEvents() | M4 | 3 | ☐ |
| 3.4 | Implement showEventsForRegion() | M4 | 3 | ☐ |
| 3.5 | Write analysis.h — declare 2 analysis functions | M5 | 3 | ☐ |
| 3.6 | Implement analyzeSupplyDisruption() | M5 | 3 | ☐ |
| 3.7 | Implement compareRegionRisk() | M5 | 3 | ☐ |
| 4.1 | Write display.h — declare 3 display functions | M6 | 4 | ☐ |
| 4.2 | Implement printRiskTable() | M6 | 4 | ☐ |
| 4.3 | Implement displayGlobalSummary() | M6 | 4 | ☐ |
| 4.4 | Implement printRiskLabel() | M6 | 4 | ☐ |
| 4.5 | Implement saveRiskScores() in data_loader.cpp | M2 | 4 | ☐ |
| 5.1 | Connect all 6 menu options in main.cpp | M1 | 5 | ☐ |
| 5.2 | Full system test — every option tested and bugs fixed | All | 5 | ☐ |
| 5.3 | Each member adds 1+ resource and 1+ event to data files | All | 5 | ☐ |
| 5.4 | Add comments above every function in all code files | All | 5 | ☐ |

# SECTION 9

Golden Rules for Your Team

## Communication

- Do a quick 10-minute check-in with your team every 2 days — even just on WhatsApp or Discord
- If you finish your task early, tell your team leader straight away
- If you are stuck for more than 30 minutes on a problem, ask your team — never stay stuck alone
- At the end of every week, share your latest code file with everyone

## Coding

- Never change another member's file without telling them first
- Every function must have a comment above it explaining what it does in plain English
- Test your code with at least 2 different inputs before saying it is done
- Always use the same struct definitions that Member 2 writes — never redefine EnergyResource
- Never put spaces in filenames — use underscores: data_loader.cpp not data loader.cpp

## When using AI tools

- Read the code the AI gives you before pasting it — make sure it matches what the task asked for
- If the AI uses libraries you don't recognise, ask it to rewrite using only standard C++
- If AI code doesn't compile, paste the error message back into the AI and ask it to fix it
- Test every function the AI writes — never trust it without testing

## Final submission checklist

| Checklist item | Done? |
|---|---|
| All 6 .h header files written and included correctly | ☐ |
| All 6 .cpp files compile without any errors | ☐ |
| Program runs with: g++ main.cpp data_loader.cpp risk_engine.cpp features.cpp analysis.cpp display.cpp -o EnergyRisk -std=c++11 | ☐ |
| All 6 menu options work correctly when tested | ☐ |
| energy_data.csv has at least 10 energy resources | ☐ |
| events.csv has at least 8 geopolitical events | ☐ |
| risk_scores_output.csv is created in data/ when option 6 is chosen | ☐ |
| Every function in every file has a comment above it | ☐ |
| At least 2 team members have tested the full program | ☐ |

**You can build this!**

This plan gives you 27 small, clear tasks — each one something a second-semester C++ student can do. Work phase by phase. Use the AI prompts when you need help with code. Communicate with your team every two days. You will have a complete, working, impressive project by the end of Week 5.